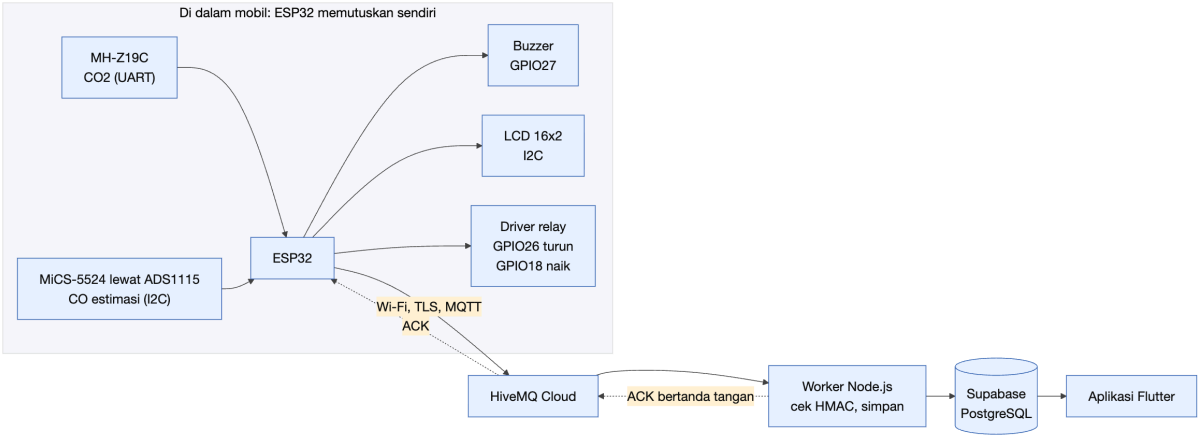
Jalur Komunikasi dan Worker AEROS

Materi belajar: jalur data ESP32 → HiveMQ → worker → database, hasil yang sudah berjalan selama pengujian, dan firmware dummy untuk uji integrasi

Firmware	v2.5 (`firmware-v2/src/main.cpp`, pesan versi 4); firmware dummy `firmware-v3-dummy/` (6 November 2026)
Worker	repo `aeros-worker` commit `efa7d8d`; isi `src/` dan `test/` sama dengan folder `worker/` di repo sensor (dibandingkan 6 November); 36 uji otomatis lulus
Sumber hasil uji	laporan uji 7 (30 Okt) dan uji 8 (3–4 Nov), `langkah-integrasi-v2.md` (29 Okt), `rencana-simulator-integrasi.md` bagian 12.8 (6 Nov)

Penanda: ✔ terbukti dari data uji atau terbaca di kode · ◐ sebagian atau sementara · ✖ belum terbukti

1. Gambaran jalur data



- ESP32 memutuskan alarm dan jendela sendiri. Jaringan hanya dipakai untuk **melaporkan** data.
- Data mengalir satu arah: ESP32 → broker MQTT (HiveMQ Cloud) → worker (Node.js) → database (Supabase/PostgreSQL) → aplikasi Flutter.
- Satu-satunya pesan balik ke ESP32 adalah **ACK**, yaitu bukti bahwa data sudah tersimpan di database.
- Perintah dari aplikasi ke ESP32 ("Saya Tahu", perintah jendela) **tidak** termasuk MVP dan sudah dibuang dari worker.

2. Komponen dan perannya

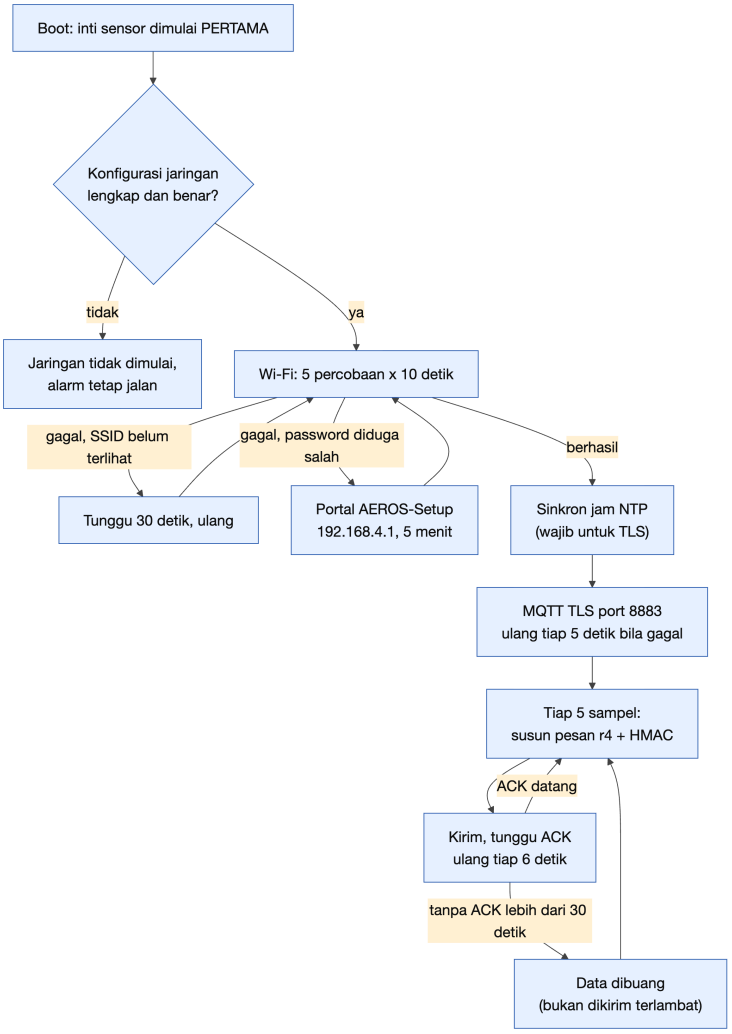
Komponen	Peran	Letak
ESP32 (`main.cpp`)	menyusun pesan tiap 5 detik, menandatangani dengan HMAC, mengirim, dan menunggu ACK	di alat
HiveMQ Cloud	broker MQTT: meneruskan pesan antara ESP32 dan worker; TLS port 8883	layanan awan
Worker (`mqtt-worker.js`)	memeriksa pesan, menulis ke database, mengirim ACK	Railway (hosting) atau laptop; **hanya satu yang boleh aktif**
Supabase	menyimpan bacaan, peringatan, dan keadaan terakhir perangkat	layanan awan
Aplikasi Flutter	membaca `device_state` dan `v_alerts`	milik tim aplikasi

Pembagian rahasia:

Rahasia	Dipakai oleh	Disimpan di
Host, username, password MQTT	ESP32 dan worker (pengguna berbeda)	ESP32: `secrets.h`; worker: `.env`
`HMAC_SECRET` (minimal 32 byte)	ESP32 dan worker, **harus sama**	keduanya
Kunci service Supabase	hanya worker	`.env` worker (tidak pernah di ESP32, aplikasi, atau git)
Sertifikat CA (ISRG Root X1)	ESP32	`root_ca.h` (publik, bukan rahasia)

3. Sisi ESP32

3.1 Alur koneksi



Tahap	Aturan
Wi-Fi	5 percobaan × 10 detik; SSID tidak terlihat → tunggu 30 detik lalu ulang; kata sandi diduga salah → portal
Portal	titik akses `AEROS-Setup`, alamat 192.168.4.1, aktif 5 menit; dibuka ulang dengan menahan tombol GPIO32 selama 5 detik
Jam	sinkron NTP sebelum TLS (sertifikat diperiksa berdasarkan waktu)

MQTT	TLS port 8883 dengan sertifikat diperiksa; percobaan ulang tiap 5 detik
Kehadiran	setelah tersambung: kirim `ONLINE` (retained). Bila koneksi putus, broker mengirim `OFFLINE` (Last Will)

3.2 Topik MQTT

Topik	Arah	Isi
`aeros/<device_id>/telemetry`	ESP32 → worker	data tiap 5 detik
`aeros/<device_id>/telemetry/ack`	worker → ESP32	bukti tersimpan, bertanda tangan
`aeros/<device_id>/status`	ESP32/broker → worker	`ONLINE` atau `OFFLINE`

Worker berlangganan `aeros/+/telemetry` dan `aeros/+/status`. Tanda `+` berarti "perangkat apa pun", sehingga ESP32 asli (`AEROS_01`) dan ESP32 dummy (`AEROS_DUMMY_01`) ditangani oleh worker yang sama.

3.3 Penyusunan satu pesan

- Sensor menghasilkan 1 sampel per detik. Setiap 5 sampel disusun menjadi 1 pesan.
- Pesan memuat nilai **terakhir** (sampel ke-5), nilai **puncak**, dan detik terjadinya puncak.

$$\text{puncak} = \max_{k=1..5} c_k, \qquad \text{detik puncak} \in \{1,2,3,4,5\}$$

$$\text{jumlah pesan} = \frac{T}{5\ \text{s}} \ \Rightarrow\ 12\ \text{pesan/menit}, \quad 720\ \text{pesan/jam}$$

- Waktu tiap pesan memakai waktu pengambilan sampel, bukan waktu kirim:

$$\text{timestamp} = t_{\text{NTP}} - \left\lfloor \frac{\text{millis}() - t_{\text{sampel}}}{1000} \right\rfloor$$

  Bila jam belum sinkron: `timestamp = 0` dan `time_valid = false`.
- Posisi jendela (estimasi timer): tertutup → `0`, terbuka → `50`, sedang bergerak → `null`.
- `event_id` (UUID) dibuat saat alarm mulai dan dipakai sampai alarm selesai. Bila CO mulai alarm saat CO_2 sudah alarm, `event_id` **diganti baru** (`trackSensorAlarm`).

3.4 Isi pesan telemetri versi 4

Field	Isi	Syarat di worker
`version`	4	3 atau 4 (3 ditolak bila `ACCEPT_V3=false`)
`device_id`	`AEROS_01` / `AEROS_DUMMY_01`	huruf, angka, `_`, `-` (maks. 48); sama dengan topik; terdaftar dan aktif
`nonce`	128 bit acak (32 karakter heksadesimal)	unik per perangkat; duplikat dianggap sudah tersimpan
`timestamp`, `time_valid`	detik Unix; 0 dan `false` bila belum NTP	maks. 90 hari lalu, maks. 5 menit ke depan
`co_ppm`, `co2_ppm`	nilai terakhir	bilangan bulat 0–100.000
`co_peak_ppm`, `co2_peak_ppm`	puncak 5 detik	≥ nilai terakhir, ≤ 100.000
`co_peak_second`, `co2_peak_second`	detik puncak	1–5
`co_level`, `co2_level`	0 normal, 1 waspada, 2 bahaya	bilangan bulat 0–2

`buzzer_state`	`on` / `off`	`on` hanya bila status bukan normal
`window_status`	0, 50, atau `null`	0–100 atau `null`
`event_id`	UUID atau `null`	wajib bila status bukan normal; harus `null` bila normal
`demo`	`true` untuk data simulasi/dummy	ditolak kecuali `ALLOW_DEMO_READINGS=true`
`signature`	HMAC-SHA256, 64 karakter heksadesimal	dibandingkan dengan perbandingan waktu-tetap

Status gabungan tidak dikirim. Worker menghitungnya:

$$\text{status} = \text{nama}\big[\max(\text{co_level},\ \text{co2_level})\big], \qquad \text{nama} = [\text{normal},\ \text{waspada},\ \text{bahaya}]$$

3.5 Tanda tangan HMAC

$$\text{signature} = \text{HMAC-SHA256}(K,\ m)$$

K = `HMAC_SECRET`; m = 17 bagian dipisah "|":

```
r4 | device_id | nonce | timestamp | time_valid(1/0) | co_ppm | co2_ppm | co_peak_ppm |
co2_peak_ppm | co_peak_second | co2_peak_second | co_level | co2_level | buzzer_state |
window_status (null = "-") | event_id (null = "-") | demo(1/0)
```

Contoh uji kontrak (kunci uji, bukan kunci asli):

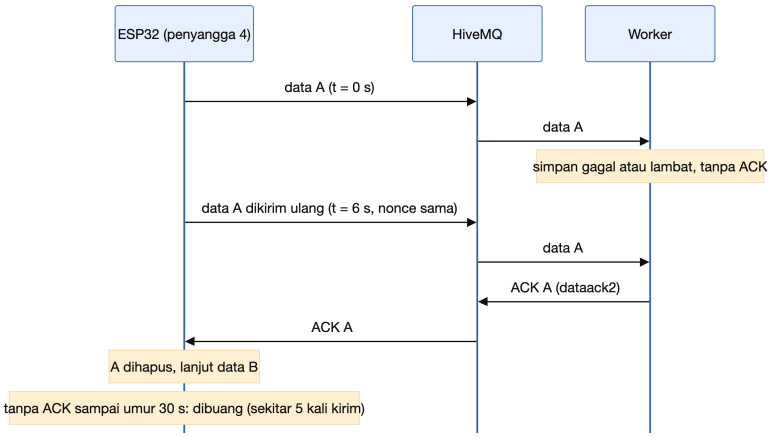
```
m =
r4|AEROS_01|0123456789abcdef0123456789abcdef|1793000000|1|12|950|15|962|2|4|0|1|on|0|216b0b86-
d596-4c56-9ae0-53394d389025|0
HMAC = 02fc8cb02d9ccde1854f18fd7ba33df4b36db52fbc56ee4bf5df1e8bf18c2418
```

- Worker menyusun ulang m dari isi pesan (`canonicalReading`) lalu membandingkan tanda tangan. Satu karakter berbeda → pesan ditolak.
- ACK juga ditandatangani:

$$\text{ACK} = \text{HMAC-SHA256}\big(K,\ \text{"dataack2|device_id|nonce|stored"}\big)$$

- HMAC membuktikan pesan berasal dari pemegang kunci dan tidak diubah. HMAC **tidak** membuktikan sensor mengukur dengan benar.

3.6 Penyangga dan kirim ulang



Parameter	Nilai	Akibat

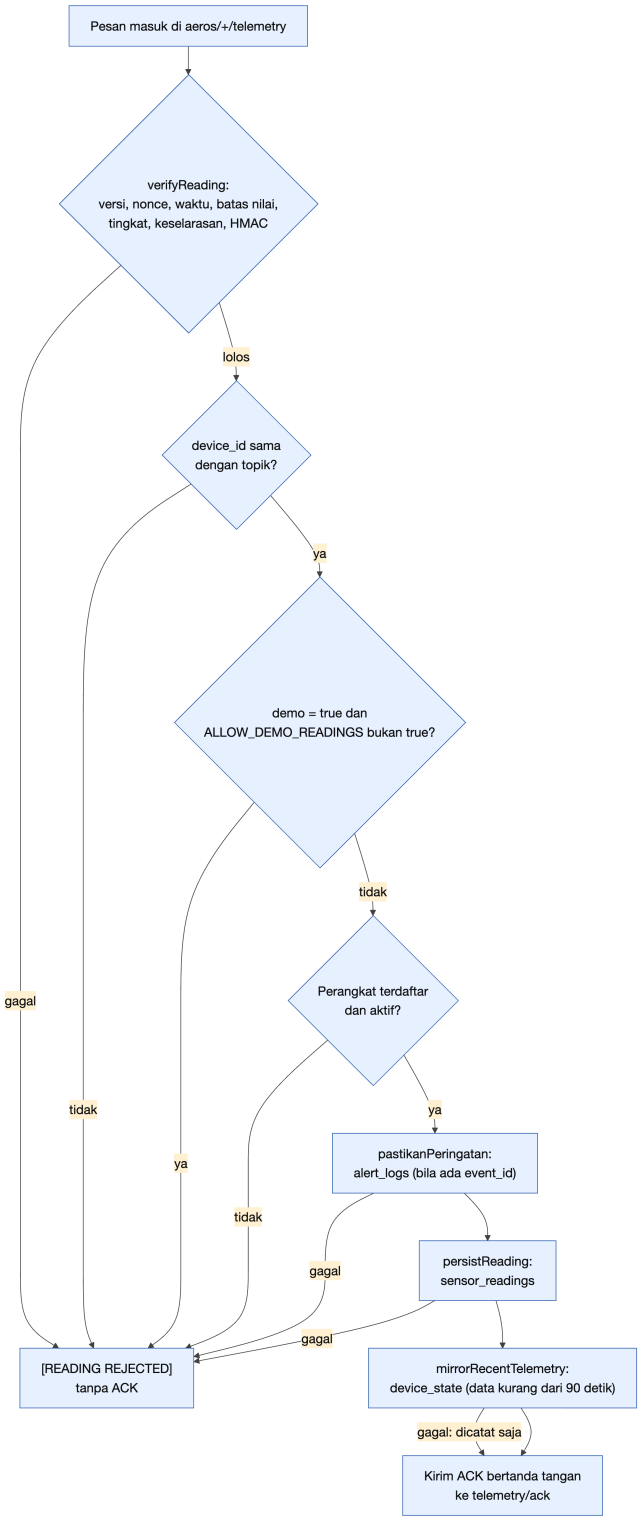
Penyangga RAM	4 pesan	penuh → pesan tertua dibuang
Kirim ulang tanpa ACK	tiap 6 detik	pesan yang sama (nonce sama) dikirim lagi
Umur maksimal pesan	30 detik	lebih tua → dibuang
MQTT terputus	—	pesan baru tidak disimpan; penyangga dikosongkan

$$\text{percobaan maksimal} \approx \left\lfloor \frac{30\ \text{s}}{6\ \text{s}} \right\rfloor = 5$$

- Pengiriman ulang tidak menggandakan data: worker mengenali nonce yang sama dan hanya mengirim ACK lagi.
- Mode online-saja dipakai sejak 29 Oktober. Antrean di flash dimatikan karena kapasitas nyata ±200 item dan pengiriman 27 detik per item tidak terkejar.

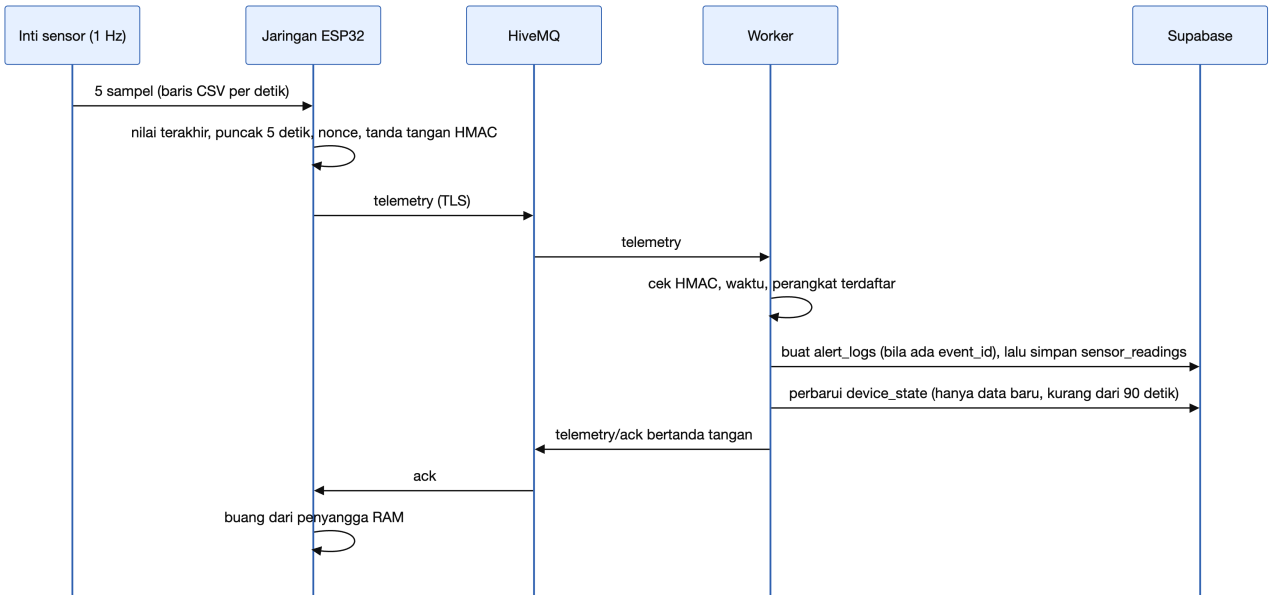
4. Worker

4.1 Pemeriksaan satu pesan



- Pesan yang gagal pada tahap mana pun dicatat `[READING REJECTED]` dan **tidak** di-ACK.
- Pesan yang lolos dicatat `[READING] <device> <nonce> new|duplicate timestamp-valid|estimated-time REAL|DEMO`.

4.2 Urutan penulisan ke database



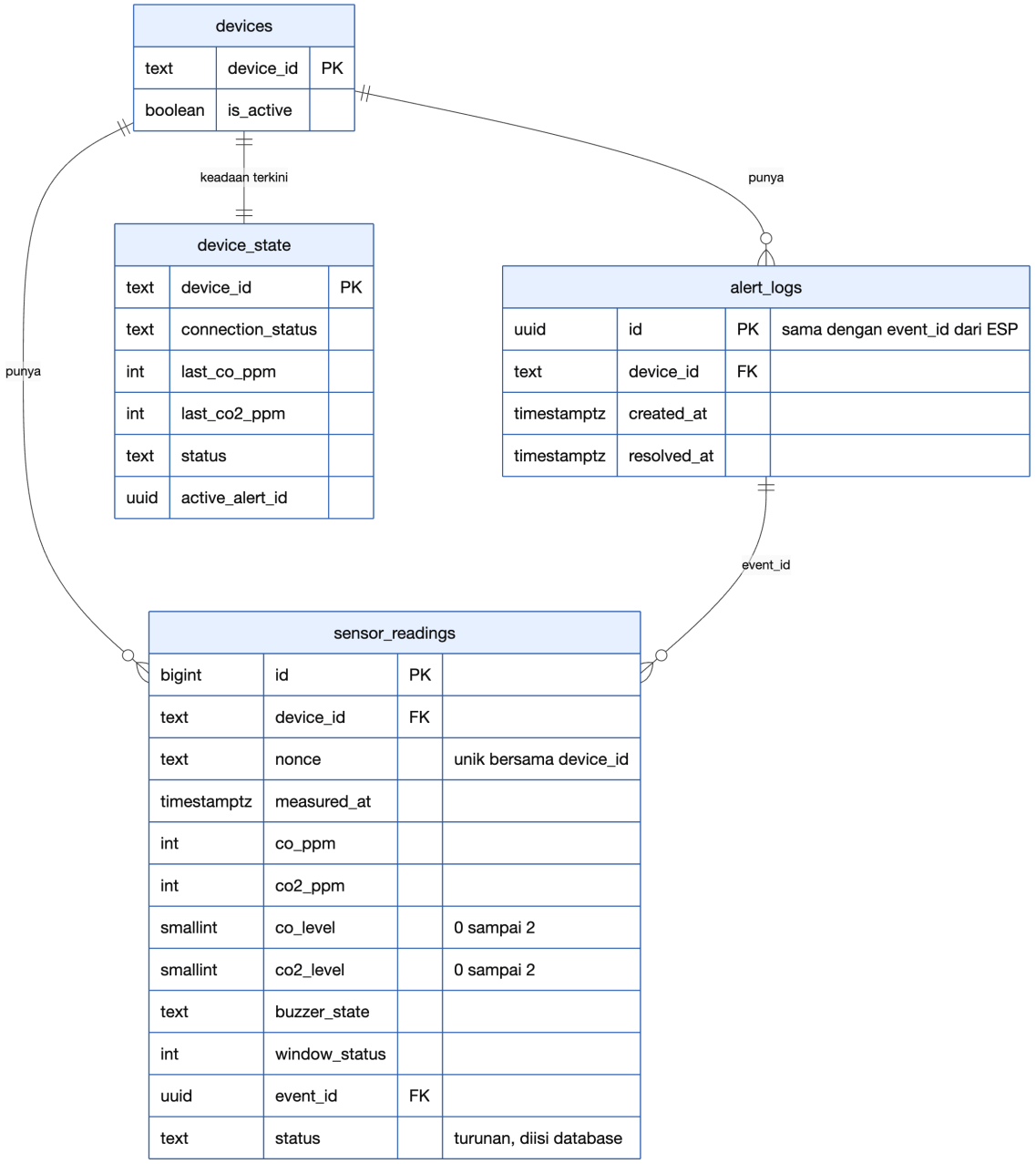
#	Langkah (fungsi)	Tabel	Bila gagal
1	`pastikanPeringatan`: sisipkan peringatan bila ada `event_id` (`id = event_id`, `created_at` = jam database)	`alert_logs`	tanpa ACK. `id` sudah ada → pesan ulang sah; milik perangkat lain → tolak
2	`persistReading`: simpan bacaan	`sensor_readings`	tanpa ACK; nonce ganda → dibuktikan ada, lalu di-ACK
3	`mirrorRecentTelemetry`: salin keadaan terakhir, tutup peringatan lama, naikkan `alert_type` ke bahaya	`device_state`, `alert_logs`	dicatat saja; ACK tetap dikirim
4	kirim ACK	—	—

- Urutan 1 sebelum 2 wajib: `sensor_readings.event_id` adalah kunci tamu ke `alert_logs`.
- Waktu bacaan: `measured_at` = `timestamp` bila `time_valid`; bila tidak, memakai jam terima worker.
- Langkah 3 hanya berjalan bila data masih baru dan merupakan bacaan terbaru perangkat:

$$\text{salin bila} \quad t_{\text{sekarang}} - \text{timestamp} \leq 90\ \text{s} \quad \wedge \quad \text{timestamp} \leq t_{\text{sekarang}} + 300\ \text{s}$$

- Peringatan ditutup (`resolved_at` diisi) saat `event_id` berubah: kembali normal atau episode baru.

4.3 Database yang ditulis



Objek	Isi
`sensor_readings`	satu baris per pesan: ppm, puncak, `co_level`, `co2_level`, buzzer, jendela, `event_id`, `measured_at`, `received_at`
`alert_logs`	satu baris per episode alarm: `id = event_id`, `created_at`, `resolved_at`, `alert_type` (sementara, demi aplikasi lama)
`device_state`	satu baris per perangkat: nilai terakhir, `status`, `active_alert_id`, `connection_status`, `updated_at`
`v_alert_gas` (view)	per peringatan per gas: tingkat maksimum, puncak ppm, waktu mulai
`v_alerts` (view)	per peringatan: tingkat dan daftar gas penyebab

$$\text{tingkat(peringatan)} = \max_{\text{bacaan dengan event_id itu}} \max(\text{co_level},\ \text{co2_level}), \qquad \text{gas} = \{\, g \mid \text{level}_g > 0 \,\}$$

4.4 Kehadiran perangkat

Pesan di `/status`	Tulisan ke `device_state`
`ONLINE`	`connection_status = online`, `last_seen_at`, `updated_at`
`OFFLINE`	`connection_status = offline`, `updated_at`

Aturan untuk aplikasi: anggap perangkat offline bila `connection_status = offline` **atau** `updated_at` lebih dari 30 detik lalu. Perangkat offline tidak boleh ditampilkan "aman".

4.5 Pengaturan worker

Variabel	Bawaan	Fungsi
`HMAC_SECRET`	wajib	sama dengan ESP32; kurang dari 32 byte → worker tidak mau berjalan
`HIVEMQ_HOST/USERNAME/PASSWORD/PORT`	port 8883	koneksi TLS (`rejectUnauthorized: true`), sambung ulang 5 detik, keep-alive 30 detik
`ALLOW_DEMO_READINGS`	bukan `true`	`true` → pesan `demo = true` diterima (wajib untuk dummy)
`WRITE_STATUS_COLUMN`	`false`	`true` hanya untuk database sebelum SQL 4b
`ACCEPT_V3`	diterima	`false` → pesan versi 3 ditolak (langkah rilis terakhir)

- Pesan versi 3 (firmware v2.1–v2.3) tidak membawa jenis gas. Worker memetakannya sementara: `co_level = 0`, `co2_level` = tingkat dari `status`. Peringatan CO dari versi 3 bisa tercatat sebagai CO_2.
- Uji otomatis (`npm test`): `security.test.js` 16, `sensor-handler.test.js` 14, `telemetry-state.test.js` 6 → 36 uji lulus.

5. Yang sudah berjalan selama pengujian

5.1 Garis waktu

Tanggal	Kegiatan	Firmware / pesan	Hasil
29 Okt	Integrasi pertama: kode komunikasi tim digabung dengan inti sensor	v2.0 / versi 3	✔ kirim dan ACK berpasangan tiap 5 detik, ACK ±0,7 detik; ✔ tanpa `secrets.h`, alarm tetap jalan
30 Okt	Uji 7: Civic, 4 orang, hotspot → HiveMQ → Railway → Supabase	v2.1 / versi 3	✔ data lengkap sampai database (5.2)
3 Nov	Worker diubah ke versi 4; SQL 4a/4b dicoba di PostgreSQL lokal	worker `efa7d8d`	✔ 36 uji lulus; ✔ galat Railway setelah 4b diperbaiki (bawaan `WRITE_STATUS_COLUMN` jadi `false`)
3–4 Nov	Uji 8: meja, ambang 700/800, ±4 jam log	v2.3 / versi 3	◐ berjalan, ada celah 19,5 menit (5.3)
6 Nov	Firmware dummy: skenario cepat (65 detik) dimainkan 2 kali ke worker Railway	v3 dummy / versi 4	✔ 2 peringatan `bahaya/{CO2}` di `v_alerts` (`c31547ad`, `eb9c5425`)
6 Nov	Firmware dummy profil bertahap diunggah ke ESP32	v3 dummy / versi 4	✔ boot, Wi-Fi, dan MQTT tersambung; ✖ skenario bertahap belum dimainkan

5.2 Uji 7 (30 Oktober): hasil sampai database

Ukuran	Hasil
Pengiriman	431 ACK untuk ±431 kiriman (18:34:25–19:10:23), tidak ada lompatan waktu
Baris database	451 baris sesi ini; puncak CO_2 di database 4025 = di log
Jeda simpan	median 1,45 s; p95 1,68 s; maks. 2,68 s
Peringatan	3 baris `alert_logs` cocok dengan 3 episode di log
`window_status`	0 saat atas, 50 saat bawah (14 baris), `null` saat bergerak (3 baris)

Perhitungan:

$$T = 19{:}10{:}23 - 18{:}34{:}25 = 2158\ \text{s} \ \Rightarrow\ \frac{2158}{5} \approx 431\ \text{pesan} \ \Rightarrow\ \text{kehilangan} = 1 - \frac{431}{431} = 0\%$$

$$\Delta t_{\text{simpan}} = \text{received_at} - \text{measured_at} \quad (\text{median } 1{,}45\ \text{s})$$

Temuan:

- `alert_type` tetap `waspada` walau CO_2 mencapai bahaya → diperbaiki di worker versi 4 (dinaikkan ke `bahaya`) dan `v_alerts.tingkat`.
- Setelah ESP32 mati, `device_state` masih berstatus waspada, tetapi `connection_status = offline` sudah benar → aturan offline aplikasi (4.4).
- Waktu tampil di **layar aplikasi** belum diukur; yang terukur hanya sampai database.

5.3 Uji 8 (3–4 November): hasil dan masalah

Ukuran	Hasil
ACK per sesi	sesi 2 = 70, sesi 3 = 68, sesi 4 = 185; sesi 1 (20:20–21:10) = 120 ACK per 10 menit
Data pertama setelah boot	MQTT 23:59:11, ACK pertama 00:00:28 (data ditahan selama pemanasan 180 detik)
Tanpa internet	4 episode alarm saat Wi-Fi mati: bunyi dan sinyal jendela tetap jalan

$$10\ \text{menit} = 600\ \text{s} \ \Rightarrow\ \frac{600}{5} = 120\ \text{pesan} = 120\ \text{ACK} \ \Rightarrow\ \text{lengkap}$$

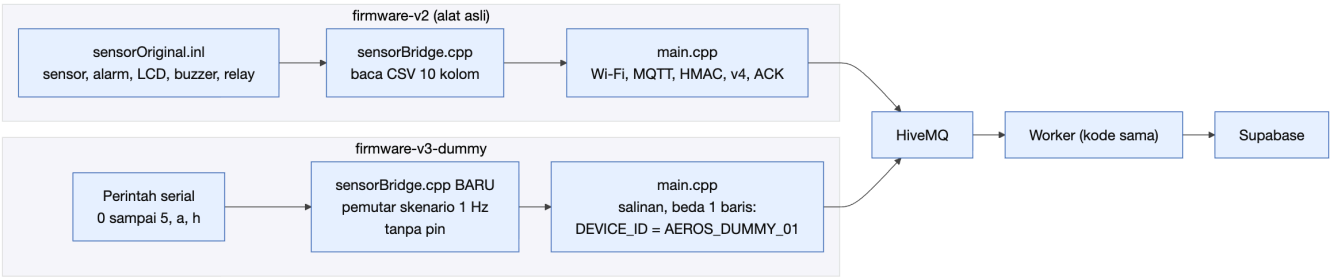
Masalah	Angka	Status
ACK berhenti padahal Wi-Fi dan MQTT tersambung (19:53–20:12)	19,5 menit; 241 pesan dibuang ($241 \times 5\ \text{s} \approx 20$ menit); 3 episode alarm jatuh di periode ini	✖ penyebab belum diketahui (dugaan sisi worker/database)
Hotspot hilang dua kali	"total dibuang 1416" ($1416 \times 5\ \text{s} \approx 2$ jam)	sesuai desain online-saja

Belum terbukti: jumlah baris di Supabase sama dengan jumlah yang dikirim, dan apakah episode 19:55–20:06 tersimpan.

6. Firmware dummy untuk uji integrasi (v3 dummy)

6.1 Tujuan dan cara kerja

- Membuktikan jalur **ESP32 asli → Wi-Fi → HiveMQ → worker → database** tanpa sensor, LCD, buzzer, atau relay, karena perangkat keras masih bermasalah.
- Hanya sumber data yang diganti. Kode pengiriman sama dengan firmware v2.5, jadi yang diuji adalah kode firmware sebenarnya.



Berkas	Di v3 dummy
`main.cpp`	salinan firmware-v2; beda **satu baris**: `DEVICE_ID = "AEROS_DUMMY_01"`
`sensorBridge.h`	sama (antarmuka tetap)
`sensorBridge.cpp`	**diganti**: pemutar skenario, 1 sampel per detik, tidak menyentuh pin apa pun
`sensorOriginal.inl`	tidak dipakai

- Semua sampel berlabel `simulated = true`, sehingga pesan selalu `demo = true`.
- `buzzer_state = on` selama ada alarm (tanpa buzzer fisik), supaya lolos aturan keselarasan worker.
- Tanpa skenario, dan setelah skenario selesai, ESP32 terus mengirim data normal (CO 0, CO_2 700) tiap 5 detik.

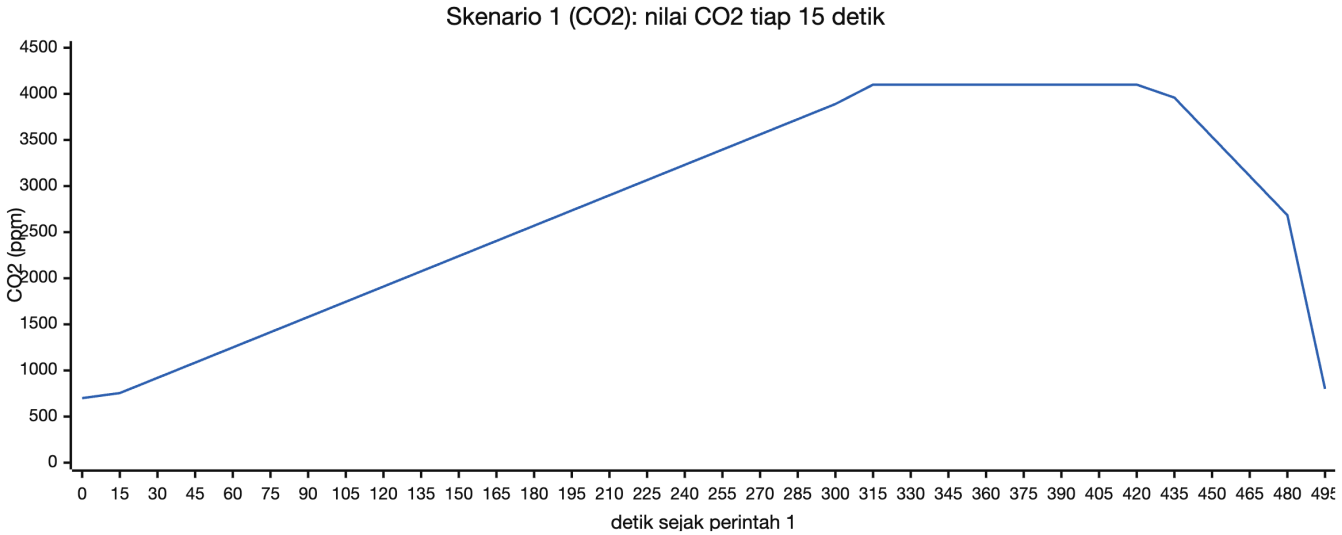
6.2 Perintah serial (115200 baud, "New Line")

Ketik	Skenario	Lama
`0`	S0 normal	40 detik
`1`	S1 CO_2 tinggi	495 detik (8,25 menit)
`2`	S2 CO tinggi	495 detik
`3`	S3a CO dan CO_2 naik bersamaan	495 detik
`4`	S3b CO_2 dulu, CO menyusul	495 detik
`5`	S4 CO_2 waspada 1 menit lalu bahaya ±2 menit	185 detik
`a`	1, 2, 3, 4 berurutan	$4 \times 495 = 1980$ s = 33 menit
`h`	bantuan	—

6.3 Profil waktu (S1 sampai S3b)

Fase	Lama	Detik ke-	Nilai	Jendela
Normal	10 s	1–10	CO 0, CO_2 700	0 (tertutup)
Naik bertahap	300 s	11–310	linier ke CO_2 4000 / CO 87	0

Bahaya	5 s	311–315	puncak CO_2 4100 / CO 90	0
Jendela turun	5 s	316–320	puncak	`null` (bergerak)
Bahaya, jendela terbuka	110 s	321–430	puncak	50 (terbuka)
Turun	60 s	431–490	linier ke CO_2 2400 / CO 30	50
Jendela naik	5 s	491–495	ke CO_2 800 / CO 0	`null`



Rumus nilai pada detik ke-i dalam satu fase sepanjang n detik (pembagian bilangan bulat):

$$c(i) = c_{\text{dari}} + \left\lfloor \frac{(c_{\text{ke}} - c_{\text{dari}}) \cdot i}{n} \right\rfloor, \qquad i = 1, \ldots, n$$

Contoh perhitungan (fase naik, n = 300, detik ke-i = detik skenario – 10):

$$\text{CO}_2:\ 700 + \left\lfloor \frac{3300 \cdot 164}{300} \right\rfloor = 700 + 1804 = 2504 \geq 2500 \ \Rightarrow\ \text{waspada di detik ke-174}$$

$$\text{CO}:\ 0 + \left\lfloor \frac{87 \cdot 121}{300} \right\rfloor = \lfloor 35{,}09 \rfloor = 35 \geq 35 \ \Rightarrow\ \text{waspada di detik ke-131}$$

$$\text{CO}_2 \text{ dan CO}:\ i = 300 \Rightarrow 4000 \text{ dan } 87 \ \Rightarrow\ \text{bahaya di detik ke-310}$$

6.4 Aturan tingkat dan `event_id`

Gas	Waspada	Bahaya	Kembali normal
CO_2	≥ 2500	≥ 4000	< 2500
CO	≥ 35	≥ 87	< 35

- Tingkat naik langsung dan **bertahan** di tingkat tertinggi sampai ppm turun di bawah batas kembali normal, lalu langsung 0.
- Dummy tidak meniru penundaan konfirmasi firmware asli (3 bacaan CO_2, 30 detik CO). Yang ditiru hanya urutan dan hasil akhirnya.
- `event_id` diatur oleh `main.cpp` yang sama: baru saat alarm mulai, baru lagi bila CO mulai saat CO_2 sudah alarm, `null` saat normal.

6.5 Hasil yang diharapkan di database (dihitung dari kode)

Skenario	Waspada mulai	Bahaya	Peringatan di `v_alerts`
S0	—	—	tidak ada; `status = normal`
S1	detik 174 (CO_2)	310–486 (177 s)	1: `bahaya`, `{CO2}`
S2	detik 131 (CO)	310–485 (176 s)	1: `bahaya`, `{CO}`
S3a	detik 131 (CO), 174 (CO_2)	310–486 (177 s)	1: `bahaya`, `{CO, CO2}`
S3b	detik 174 (CO_2), 239 (CO)	310–486 (177 s)	2: E1 `waspada/{CO2}` (174–239); E2 `bahaya/{CO, CO2}`
S4	detik 1 (CO_2 2600)	61–182 (122 s)	1: `bahaya`, `{CO2}`

$$\text{baris } \texttt{sensor_readings} \text{ per skenario} \approx \frac{495}{5} = 99$$

- Satu pesan bisa memuat sampel dari dua fase, karena `main.cpp` menggabung 5 sampel tanpa tahu kapan skenario dimulai. Keduanya tetap lolos pemeriksaan worker.
- S3b menghasilkan dua kartu peringatan di aplikasi; ini perlu dikonfirmasi ke tim aplikasi.

6.6 Prasyarat sebelum dimainkan

Prasyarat	Nilai
`secrets.h` ESP32	Wi-Fi 2,4 GHz, pengguna MQTT ESP, `HMAC_SECRET` sama dengan worker
Tabel `devices`	`AEROS_DUMMY_01` terdaftar, `is_active = true`
Worker	`ALLOW_DEMO_READINGS=true` selama uji, lalu dikembalikan; `WRITE_STATUS_COLUMN` sesuai database; hanya satu worker aktif
Serial Monitor	tunggu `[BIRTH] ONLINE berhasil dikirim` sebelum mengetik perintah
Bukti	log worker `[READING] ... DEMO` tanpa `[READING REJECTED]`; baris di `v_alerts` dan `v_alert_gas`; tangkapan layar aplikasi

6.7 Perbedaan dokumen dan kode dummy

> Kode `sensorBridge.cpp` (diubah 6 Nov 02:30) lebih baru daripada `firmware-v3-dummy/README.md` dan `rencana-simulator-integrasi.md`. Materi ini mengikuti **kode**.
>
> | Hal | README / rencana | Kode |
> |---|---|---|
> | Saat diam | ESP32 tidak mengirim data sampai ada perintah, lalu berhenti 10 detik setelah skenario selesai | terus mengirim data normal (CO 0, CO_2 700) |
> | Perintah `s` (berhenti) | ada | tidak ada |
> | Skenario 5 | tidak disebut | ada (S4 di materi ini) |
> | Lama skenario | 505 s, dengan fase normal terakhir 10 s | 495 s, tanpa fase normal terakhir |
> | Perintah `a` | ±34 menit | 33 menit |

6.8 Batasan dummy

- Membuktikan jalur dan kode pengiriman firmware, **bukan** sensor, buzzer, relay, atau jendela.

- Data tercatat `DEMO` dan harus disampaikan sebagai simulasi: "*Ini simulasi jalur data karena hardware belum siap. Pesannya memakai format dan tanda tangan yang sama dengan firmware, dan data tercatat berlabel DEMO.*"
- Saat MQTT putus, data tidak disimpan (sama dengan firmware asli).

7. Ringkasan

Hal	Status
ESP32 → HiveMQ → worker → Supabase, data nyata (uji 7)	✔ 0% hilang, jeda simpan median 1,45 s
ACK hanya setelah tersimpan; kirim ulang tanpa data ganda	✔ kode dan uji otomatis
Alarm dan jendela tetap jalan tanpa internet	✔ uji 8
Pesan versi 4 dari ESP32 sampai `v_alerts`	◐ hanya skenario cepat dummy (2 peringatan CO_2)
Skenario bertahap S0–S4 di ESP32, termasuk CO dan dua episode (S3b)	✖ belum dimainkan
Penyebab celah ACK 19,5 menit (uji 8)	✖ belum diketahui
Jumlah baris database = jumlah terkirim (uji 8)	✖ belum dibandingkan
Waktu tampil di layar aplikasi	✖ belum diukur
Firmware v2.5 dengan pesan versi 4 dari sensor asli	✖ belum diuji